# Implementação e Validação em Hardware do Processador MIPS Monociclo com Instruções de Desvio

## Segunda Entrega: Síntese em FPGA Intel MAX 10, Simulação RTL e Verificação Estrutural

**José Dercy Vieira da Silva Filho**
**Vinicius Menezes**
**Isabela Furtado**

Faculdade de Engenharia Elétrica e da Computação – FEEC
Universidade Federal do Amazonas – UFAM
Manaus, Brasil

Arquitetura de Sistemas Digitais e Laboratório – 2026/2

# Agenda da Apresentação

## Arquitetura e Módulos do Caminho de Dados

- **Visão Geral da Arquitetura Monociclo**
- **ULA de 32 bits:** Otimização e Flags
- **Banco de Registradores:** Temporização e Reg $0
- **Unidade de Controle:** Decodificação de Saltos
- **Subsistema de Memória:** Modelo Harvard
- **Integração Top-Level:** Cascata do Próximo PC

## Validação em Hardware e Metodologia

- **Bateria de Testbenches:** Verificação formal
- **Formas de Onda no GTKWave:** Demonstração prática
- **Síntese no Intel Quartus Prime:** FPGA MAX 10
- **Análise Temporal e Caminho Crítico**
- **Cumprimento das 5 Rubricas da Entrega**
- **Transição para o Pipeline (Entrega 03)**

# Visão Geral da Arquitetura MIPS Monociclo

## Princípios Fundamentais do Processador

- **Ciclo Único ($CPI = 1$):** Toda instrução é buscada, decodificada, executada e gravada no intervalo de exatamente um período de clock;
- **Largura de Dados e Endereçamento:** Barramentos de dados de 32 bits com memória endereçada por byte e alinhada por palavra (4 bytes);
- **Conjunto de Instruções Suportado:**
  - *Tipo-R:* `add`, `sub`, `and`, `or`, `slt`, `nor`, `jr`;
  - *Tipo-I:* `addi`, `lw`, `sw`, `beq`;
  - *Tipo-J:* `j`, `jal`.
- **Sem Conflitos Estruturais:** Memórias fisicamente segregadas para instruções e dados (Arquitetura Harvard).

## Fluxo de Execução das Instruções

1. **Busca (IF):** Leitura da ROM indexada por `PC[31:2]`;
2. **Decodificação (ID):** Acesso ao RegFile e geração dos sinais de controle;
3. **Execução (EX):** Cálculo aritmético/lógico na ULA ou soma de endereço;
4. **Memória (MEM):** Acesso à memória de dados (`lw` / `sw`);
5. **Escrita (WB):** Atualização do RegFile e comutação do próximo PC.

# ULA de 32 bits: Arquitetura e Otimização Lógica

## Somador/Subtrator Compartilhado (Harris & Harris)

- **Minimização de Área em Silício/FPGA:** Evita sintetizar blocos distintos de soma e subtração. Utiliza uma única cadeia somadora de 32 bits:

$$B_{\text{inv}} = B \oplus \text{alu_control}[2], \quad S = A + B_{\text{inv}} + \text{alu_control}[2]$$

- Quando alu_control[2] $= 1$, as portas XOR invertem os bits de $B$ e o bit atua como $C_{\text{in}} = 1$ (Complemento de Dois: $\overline{B} + 1$);
- **Geração da Flag Zero:** Redução unária flag_zero $= \sim(|\,\text{alu_result})$ via árvore balanceada de portas NOR ($O(1)$ lógico);
- **SLT Assinado:** Avalia o bit de sinal com tratamento estrito de overflow aritmético.

## Mapeamento de Operações da ULA

| alu_control[2:0] | Função | Descrição |
|---|---|---|
| 000 | AND | Lógico bit a bit |
| 001 | OR | Lógico bit a bit |
| 010 | ADD | Soma com sinal |
| 110 | SUB | Subtração ($A - B$) |
| 111 | SLT | Comparação com sinal |
| 100 | NOR | Lógico bit a bit |

**SLT Assinado:** Avalia o bit de sinal com correção estrita de overflow aritmético.

# ULA de 32 bits: Verificação Formal e Síntese Física

## Bateria Formal de Testes Unitários (`tb_alu.v`)

- **11 Casos de Teste Automatizados:**
  - Operações lógicas básicas (`AND`, `OR`, `NOR`);
  - Adição de valores positivos e negativos;
  - Subtração resultando em valores positivos e negativos;
  - Subtração de operandos iguais com teste da `flag_zero`;
  - Casos de borda da comparação assinada `SLT` (positivo vs. negativo, magnitudes opostas).
- **Assertivas Formais:** Checagem automática via `$display` e parada imediata sob discrepância.

## Síntese no Intel Quartus Prime 19.1 Lite

- **Dispositivo FPGA:** Intel MAX 10 (`10M50DAF484C7G`);
- **Placa Alvo:** Terasic DE10-Lite;
- **Mapeamento Físico:**
  - 0 erros de compilação (`make quartus-build`);
  - Mapeamento ótimo em Logic Elements (LEs) de 4 entradas;
  - Árvore de multiplexação combinacional balanceada.
- **Observabilidade RTL:** Estrutura confirmada no Netlist RTL Viewer.

# Banco de Registradores de 32 bits (`reg_file.v`)

## Topologia Estrutural do Register File

- **Capacidade:** 32 registradores de uso geral de 32 bits ($0 a $31);
- **Portas de Leitura (2 Portas):**
  - Endereços: `raddr1[4:0]` e `raddr2[4:0]`;
  - Saídas de dados: `rdata1[31:0]` e `rdata2[31:0]`;
  - **Modo de Leitura:** Assíncrona / combinacional pura.
- **Porta de Escrita (1 Porta):**
  - Endereço de destino: `waddr[4:0]`;
  - Dado de entrada: `wdata[31:0]`;
  - Habilitação: `reg_write` síncrono em posedge `clk`.

## Garantia Estrutural do Registrador $0

- **Regra Canônica da Arquitetura MIPS:** O registrador $0 armazena permanentemente a constante zero;
- **Amarração em Hardware (*Hardwired Zero*):**

```
rdata = (raddr == 5'b0) ? 32'b0 :
            regs[raddr];
```

- Qualquer tentativa de escrita em $0 é sumariamente ignorada pelo hardware, blindando a arquitetura contra desvios de software.

# Unidade de Controle e Decodificação de Instruções

## Decodificação Hierárquica em Dois Níveis

- **Nível 1: Controle Principal (`Opcode[5:0]`):** Decodifica o formato da instrução e estabelece os sinais gerais de controle:
  - `RegDst`, `ALUSrc`, `MemtoReg`, `RegWrite`;
  - `MemRead`, `MemWrite`, `Branch`, `Jump`, `Jal`;
  - Código de operação básica para a ULA: `ALUOp[1:0]`.
- **Nível 2: ALU Control (`ALUOp[1:0]` + `Funct[5:0]`):** Define a operação exata da ULA (`alu_control[2:0]`) e sinaliza retorno de procedimento (`JR_Control`).

## Mapeamento de `ALUOp`

| ALUOp | Instruções | Operação ULA |
|---|---|---|
| 00 | `lw, sw, addi` | Adição (010) |
| 01 | `beq` | Subtração (110) |
| 10 | Tipo-R | Decodifica `Funct` |

**Modularidade:** Isola o decodificador de funções aritméticas do fluxo de controle geral do datapath.

# Decodificação dos Desvios: `beq`, `j`, `jal` e `jr`

## Matriz de Controle para Instruções de Salto

| Instrução | Tipo | Branch | Jump | Jal | JR_Ctrl | RegDst | RegWrite | Destino do PC |
|---|---|---|---|---|---|---|---|---|
| `beq` | I | **1** | 0 | 0 | 0 | X | 0 | $PC + 4$ ou $PC_{\text{branch}}$ |
| `j` | J | 0 | **1** | 0 | 0 | X | 0 | $\{PC + 4[31:28], \text{instr}[25:0], 2'b00\}$ |
| `jal` | J | 0 | **1** | **1** | 0 | **2'b10** ($31) | **1** | $\{PC + 4[31:28], \text{instr}[25:0], 2'b00\}$ |
| `jr` | R | 0 | 0 | 0 | **1** | X | 0 | rdata1 ($rs) |

## Particularidades de `jal` (Call)

- Salta para o endereço alvo do tipo J;
- Salva o endereço de retorno ($PC + 4$) no registrador `$31` (`$ra`);
- Multiplexador `mem_to_reg` seleciona $PC + 4$.

## Particularidades de `jr` (Return)

- Identificado pelo código de função: `Funct = 0x08`;
- Sinal `JR_Control` comuta o multiplexador do PC;
- Restaura o fluxo para o endereço lido de `$ra`.

# Subsistema de Memória: Instruções (ROM) e Dados (RAM)

## Memória de Instruções (`instr_mem.v`)

- **Tipo:** ROM assíncrona;
- **Capacidade:** 256 palavras de 32 bits (1 KB);
- **Endereçamento:** Indexada por `addr[31:2]` (ignora os 2 bits menos significativos para alinhamento em palavras de 4 bytes);
- **Inicialização:** Carregamento estático do código executável via diretiva `$readmemh`.

## Memória de Dados (`data_mem.v`)

- **Tipo:** RAM síncrona / combinacional;
- **Capacidade:** 256 palavras de 32 bits (1 KB);
- **Porta de Leitura:** Assíncrona para acesso rápido no mesmo ciclo pela instrução `lw`;
- **Porta de Escrita:** Síncrona na subida do clock (`posedge clk`) sob ativação de `mem_write` (`sw`).

## Vantagem da Separação Harvard no Processador Monociclo

A existência de barramentos físicos distintos para busca de código e acesso a dados permite que instruções de memória como `lw` realizem a busca da instrução e a leitura de dados no mesmo período de clock, eliminando conflitos de barramento.

# Fiação Estrutural do Top-Level e Cascata do Próximo PC

## Cascata Ordenada de Multiplexadores do PC

A definição do próximo valor do Program Counter (`next_pc`) obedece a uma hierarquia estrita de prioridades:

1. **Nível 1 (Branch Condicional):**

$$PC_{\text{seq}} = (\text{branch} \wedge \text{zero}) \ ? \ PC_{\text{branch}} : PC + 4$$

2. **Nível 2 (Jump Incondicional / Call):**

$$PC_{\text{j_sel}} = \text{jump} \ ? \ \{PC{+}4[31:28], \text{instr}[25:0], 2'b00\} : PC_{\text{seq}}$$

3. **Nível 3 (Return por Registrador – Prioritário):**

$$PC_{\text{next}} = \text{jr_ctrl} \ ? \ \text{rdata1} : PC_{\text{j_sel}}$$

## Interconexão dos Subsistemas

- **`instr_mem`:** Fornece a instrução lida;
- **`control_unit`:** Gera todos os sinais de controle;
- **`datapath`:** Conecta PC, somadores, RegFile, ULA e árvore de multiplexação;
- **`data_mem`:** Executa operações de carga e armazenamento;
- **Observabilidade Externa:** Sinais essenciais de depuração expostos nas portas do Top-Level.

# Validação Comportamental do Top-Level no GTKWave

## Programa de Validação e Roteiro de Teste

1. **Inicialização:** Instruções `addi` para carregar valores base nos registradores ($s0 a $s3);
2. **Aritmética e Lógica:** Execução de `add` e `sub` verificando a propagação na ULA;
3. **Desvio Condicional (`beq`):** Avaliação de salto tomado ($A == B$) e não tomado ($A \neq B$);
4. **Chamada de Função (`jal`):** Salto para o endereço da sub-rotina e gravação de $PC + 4$ em $31 ($ra);
5. **Retorno de Função (`jr`):** Retorno imediato via `jr $ra` retomando o fluxo principal.

## Sinais Analisados no GTKWave (`make waves-top`)

- `clk`, `rst`: Sinais de sincronismo e inicialização;
- `pc[31:0]`: Evolução sequencial e desvios abruptos;
- `instr[31:0]`: Código da instrução corrente em hexadecimal;
- `alu_result[31:0]` e `flag_zero`: Resultados operacionais;
- `jump`, `jal`, `jr_ctrl`: Confirmação de disparo dos controles de desvio;
- `rdata1[31:0]`: Valor de $ra lido durante a execução do `jr`.

# Resultados da Síntese Física no FPGA Intel MAX 10

## Síntese no Intel Quartus Prime 19.1 Lite

- **Dispositivo:** Intel MAX 10 (`10M50DAF484C7G`);
- **Placa Alvo:** Terasic DE10-Lite;
- **Compilação Física:** 0 erros e 0 *critical warnings* no fluxo Analysis & Synthesis, Fitter e Assembler;
- **Bitstream Gerado:** Arquivo `quartus/output_files/mips_monociclo.sof` pronto para gravação JTAG.

## Caminho Crítico do Processador Monociclo

Delimitado pela instrução mais longa (`lw`):

$$T_{clk} \geq T_{ROM} + T_{RegFile} + T_{ALU} + T_{RAM} + T_{Mux} + T_{Setup}$$

## Governança e Boas Práticas HDL

- **Parametrização Rigorosa (Rubrica 4):** Todos os 15 módulos HDL utilizam constantes `parameter`;
- **Automação Total via Makefile:** 15 alvos operacionais cobrindo síntese, simulação, visualização de ondas e compilação de relatórios;
- **Verificação em Bateria:** Testbenches formais para todos os submódulos e integração do Top-Level.

# Conclusão e Próximos Passos: Rumo ao Pipeline

## Atendimento Integral das 5 Rubricas da Entrega 02

- ✓ **Rubrica 1 – Ambiente de Desenvolvimento:** Fluxo Quartus Prime, Icarus Verilog, GTKWave e Tectonic operando;
- ✓ **Rubrica 2 – Caminho de Dados com Jumps:** Saltos `j`, `jal`, `jr` e `beq` integrados e funcionais;
- ✓ **Rubrica 3 – Bateria de Testbenches:** Verificação formal cobrindo ULA, Registradores, Controle, Memória e Top-Level;
- ✓ **Rubrica 4 – Código 100% Parametrizado:** Padronização limpa em todos os fontes Verilog;
- ✓ **Rubrica 5 – Apresentação e Relatório:** Slides em alta resolução e relatório no padrão IEEE compilado.

## Transição Arquitetural para a Entrega 03

A separação modular e a fiação limpa desenvolvidas estabelecem a fundação ideal para a transformação no processador com **Pipeline de 5 estágios** (IF, ID, EX, MEM, WB):

- Inserção dos registradores de estágio;
- Unidade de Detecção de Conflitos (*Hazard Detection Unit*);
- Unidade de Encaminhamento (*Forwarding Unit*).